

DS PROJECT DOCUMENTATION

Generated: May 18, 2026

1. LIBRARIES USED & PURPOSE

Library	Version	Purpose
pandas	Latest	Data manipulation and analysis. Used for loading, cleaning, and processing CSV datasets
numpy	Latest	Numerical computing library. Provides support for large, multi-dimensional arrays and math
scikit-learn	Latest	Machine learning library. Used for TfidfVectorizer (text feature extraction) and PassiveAggr
nltk	Latest	Natural Language Processing. Used for downloading stopwords and text preprocessing (re
streamlit	Latest	Web application framework. Builds interactive user interface for the fake news detection ap
pillow (PIL)	Latest	Image processing library. Analyzes images for suspicion scores by computing entropy, colo

2. MACHINE LEARNING MODEL

Model Type: PassiveAggressiveClassifier
Source: scikit-learn library
Purpose: Binary classification for fake news detection

- Why PassiveAggressiveClassifier?**
- Efficient for large-scale text classification tasks
 - Suitable for online learning with streaming data
 - Fast training and prediction times
 - Memory-efficient compared to other algorithms like SVM
 - Works well with TF-IDF vectorized text features

- Model Configuration:**
- Max iterations: 50
 - Random state: 42 (for reproducibility)

- Feature Extraction:**
- TfidfVectorizer with max_features=5000
 - N-gram range: (1, 2) - unigrams and bigrams
 - Sublinear TF scaling for better feature representation

- Training Data Split:**
- Training set: 80%
 - Test set: 20%
 - Stratified split to maintain class distribution

3. TEXT PREPROCESSING PIPELINE

The `clean_text()` function in `preprocess.py` performs the following steps:

1. **Lowercase Conversion:** Convert all text to lowercase for uniform comparison
2. **URL Removal:** Remove HTTP/HTTPS URLs and www links
3. **HTML Tag Removal:** Strip HTML tags from text
4. **Special Character Removal:** Keep only alphanumeric characters and spaces
5. **Whitespace Normalization:** Replace multiple spaces with single space
6. **Stopword Removal:** Remove common English words using NLTK corpus
7. **Tokenization:** Split cleaned text into individual tokens

NLTK Resources: The application automatically downloads English stopwords on first run.

4. IMAGE ANALYSIS FEATURES

The `image_utils.py` module analyzes images to compute suspicion scores based on visual characteristics:

Features Extracted:

- **Entropy:** Measures randomness/disorder in image (lower values = more uniform, potentially suspicious)
- **Unique Colors:** Number of distinct colors in image
- **Standard Deviation:** Variation in pixel values across color channels
- **Aspect Ratio:** Width-to-height ratio to detect distorted images

Scoring Logic:

- Base score: 0.2
- Low entropy (<4.0) adds 0.25 to score (indicates unnatural image)
- Medium entropy (4.0-5.0) adds 0.1 to score
- Images with <256 unique colors receive additional suspicion points

5. DATABASE USAGE

Database Type: CSV Files (Comma-Separated Values)

Why CSV and Not a Relational Database?

- **Simplicity:** CSV files are lightweight and easy to work with for machine learning projects
- **Data Science Workflow:** pandas library natively supports CSV I/O operations
- **Deployment:** Easier to deploy on platforms like Heroku (as indicated by Procfile)
- **Version Control:** CSV files can be tracked in Git for data versioning
- **Portability:** CSV is a universal format, compatible with any tool or language
- **Dataset Size:** Training dataset fits in memory; database overhead not needed

Dataset Files:

- **Primary:** data/fake_news_dataset.csv
- **Archive:** Additional CSV files in 'archive/' directory for extended training

Data Structure:

- **news/text column:** Article content
- **title column:** Article headline (optional, concatenated with text)
- **label column:** Binary classification (0=Fake, 1=Real)

PROJECT SUMMARY

This **Fake News Detection** project is a machine learning application that combines text analysis and image processing to identify fake news articles.

Workflow:

1. Load news dataset from CSV files
2. Preprocess text using NLTK and regex techniques
3. Extract features using TfidfVectorizer
4. Train PassiveAggressiveClassifier model
5. Deploy as Streamlit web application
6. Accept user input (text and/or images) for real-time predictions
7. Return classification results and suspicion scores

Key Technologies Stack:

- **Backend:** Python with scikit-learn for ML
- **Frontend:** Streamlit for interactive UI
- **Data Processing:** pandas and numpy
- **NLP:** NLTK for text preprocessing
- **Image Processing:** Pillow for image analysis
- **Deployment:** Heroku-ready (Procfile included)